SCC: A Safety Case Compiler for Autonomous Drones

Andre Oliveira
RNFC Project
handre.oliveira@gmail.com

***Abstract*—Safety cases for autonomous drones are typically static documents, disconnected from the runtime behaviour they are meant to assure. We present SCC (Safety Case Compiler), a tool that takes safety requirements in a small DSL and compiles them into: (i) executable Rust test stubs, (ii) Goal Structuring Notation (GSN) nodes, and (iii) cryptographic proofs in a hash chain. One source of truth (the requirements file) generates three verified artefacts. We demonstrate SCC on a 3-requirement UAV envelope (speed, altitude, obstacle clearance), showing that a single-line change regenerates all artefacts and re-validates the chain. SCC is implemented in Rust (`no_std` compatible) and evaluated on 10 unit tests and 1 end-to-end demo.**

***Index Terms*—safety case, certification, GSN, Rust, UAV, DO-326A**

I. Introduction

Safety cases are the primary mechanism by which aviation authorities (EASA, FAA) assess whether an autonomous system is acceptably safe. A safety case is a structured argument, typically expressed in Goal Structuring Notation (GSN) [1], that links high-level safety goals to concrete evidence. For unmanned aerial vehicles (UAVs), the safety case must cover the entire envelope: speed, altitude, obstacle clearance, and failure modes.

The problem is that safety cases are *static*. They are written once, reviewed once, and then disconnected from the runtime behaviour they are meant to assure. When the drone flies, nothing in the safety case changes — even if a sensor degrades, a motor fails, or the operator sends an unsafe command. The evidence is frozen in time.

This disconnect has two consequences. First, the safety case cannot detect when the system drifts outside the envelope it was certified for. Second, there is no mechanism to *regenerate* the safety case when requirements change. A single-line change in a requirement (e.g., `speed_max: 10.0` to `speed_max: 8.0`) requires manual updates to the tests, the GSN nodes, and the audit trail — a process that is error-prone and rarely performed.

We present SCC (Safety Case Compiler), a tool that compiles safety requirements into executable evidence. SCC takes a small DSL of requirements (speed, altitude, obstacle clearance) and produces three artefacts per requirement:

1) A Rust test stub that verifies the invariant at runtime.
2) A GSN node that links the requirement to its evidence.
3) A cryptographic block in a hash chain (CertChain) that records the compiled requirement.

The key property is *single source of truth*: changing one line in the requirements file regenerates all three artefacts and re-validates the chain. There is no manual synchronisation, and no possibility of the GSN node disagreeing with the test.

We evaluate SCC on a 3-requirement UAV envelope (speed ≤ 10 m/s, altitude in $[5, 120]$ m, obstacle clearance ≥ 2 m). The compiler produces 3 tests, 3 GSN nodes, and a 3-block chain in under 100 ms. A single-line change regenerates all artefacts and the chain re-validates. SCC is implemented in Rust (`no_std` compatible) and tested with 10 unit tests.

The contributions of this paper are:

- A DSL for safety requirements that compiles to executable evidence (tests + GSN + chain blocks).
- A compiler architecture that guarantees consistency between the three artefacts.
- An integration with a hash chain that makes the safety case tamper-evident and auditable.
- An evaluation on a UAV envelope, including a demonstration that a single-line change regenerates all artefacts.

The paper is organised as follows. Section II reviews safety cases and runtime verification. Section III presents the SCC design. Section IV describes the implementation. Section V evaluates SCC on the UAV envelope. Section VI discusses the relation to DO-326A and open problems. Section VII concludes.

II. Background

A. Safety Cases and GSN

A safety case is a structured argument that a system is acceptably safe. The Goal Structuring Notation (GSN) [1], [2] is the de facto standard for expressing such arguments. A GSN argument is a directed acyclic graph of goals (claims), strategies (reasoning steps), and solutions (evidence). For UAVs, EASA SC-VTOL [3] and DO-178C [4] define the regulatory context.

The limitation of GSN in practice is that solutions are static artefacts: a test report, a simulation log, a hardware-in-the-loop trace. When the system changes, the solutions become stale, but the GSN argument is rarely updated. This disconnect is the problem SCC addresses.

B. Runtime Verification

Runtime verification [5] monitors a running system against a formal specification and raises an alarm when the specification is violated. It is complementary to static safety cases: the

safety case argues *why* the system should be safe; runtime verification checks *whether* it is.

SCC bridges the two: it compiles a safety requirement into both a runtime monitor (a Rust test) and a GSN node, ensuring that the argument and the monitor are generated from the same source.

C. Hash Chains for Audit

A hash chain [6], [7] is a sequence of blocks where each block contains the hash of the previous one. Any modification to an earlier block invalidates all subsequent hashes, making tampering detectable. SCC uses a hash chain to record each compiled requirement, providing a tamper-evident audit trail that a regulator can verify independently of the operator.

III. SCC Design

A. Requirements DSL

SCC accepts a small domain-specific language for UAV safety requirements. Each line has the form:

```
<name>: <value(s)> <severity>
```

where `severity` is one of `HIGH`, `MEDIUM`, `LOW`. Three requirement types are supported:

- `speed_max`: a scalar upper bound on speed.
- `altitude_range`: a lower and upper bound on altitude.
- `obstacle_clearance`: a minimum distance to obstacles.

Example:

```
speed_max: 10.0 HIGH
altitude_range: 5.0 120.0 HIGH
obstacle_clearance: 2.0 MEDIUM
```

B. Compilation Pipeline

The pipeline has four stages:

1) **Parse**: the DSL is parsed into a list of `Requirement` structs. Malformed lines are rejected with a line number and reason.
2) **Generate tests**: each requirement becomes a Rust test stub that asserts the invariant.
3) **Generate GSN**: each requirement becomes a GSN node with severity, evidence reference, and chain block index.
4) **Compile chain**: each requirement becomes a block in a SHA-256 hash chain, binding the requirement to the mission context.

The output is three artefacts per requirement, all derived from the same input. There is no manual synchronisation.

C. Why Rust

Rust was chosen for three reasons: (i) the flight control stack (`rnfc-core`) is already in Rust, so SCC integrates natively; (ii) the `no_std` subset allows SCC to run on the companion computer without a full OS; (iii) the type system makes the parser and the code generator easy to test exhaustively.

IV. Implementation

A. Crate Structure

SCC is implemented as the crate `rnfc-scc` (160 lines of Rust). The public API has four functions:

- `parse_requirements(input: &str) -> Result<Vec<Requirement>, String>`
- `generate_test(req: &Requirement) -> String`
- `generate_gsn(req: &Requirement, block: usize) -> String`
- `compile_requirements(reqs: &[Requirement]) -> Result<CertChain, String>`

The crate depends on `rnfc-certchain` (SHA-256 hash chain) and uses `sha2` for the hash function.

B. Integration with CertChain

Each requirement is hashed together with its values and severity, and the hash is appended to a `CertChain`. The chain is deterministic: the same requirements always produce the same chain root. This allows a regulator to verify that a drone was certified against a specific set of requirements, and that the requirements have not changed since certification.

V. Evaluation

A. Case Study: 3-Requirement UAV Envelope

We evaluate SCC on the envelope: `speed_max = 10.0 m/s`, `altitude_range = [5.0, 120.0] m`, `obstacle_clearance = 2.0 m`. All three are `HIGH` or `MEDIUM` severity.

B. Results

The compiler produces:

- 3 Rust test stubs (one per requirement), each with the assertion and the severity as a comment.
- 3 GSN nodes, each with the requirement, severity, evidence reference (`test_<name>_invariant`), and chain block index.
- A 3-block `CertChain` with a deterministic root hash.

A single-line change (`speed_max: 10.0` to `8.0`) regenerates all three artefacts and produces a different chain root. The change is therefore visible in the audit trail.

C. Limitations

SCC supports only three requirement types. It does not support quantifiers (*for all* trajectories), temporal operators (*always*, *eventually*), or composition of safety cases. The physics used to evaluate the requirements is synthetic (1D/2D), not 6-DOF. These are open problems.

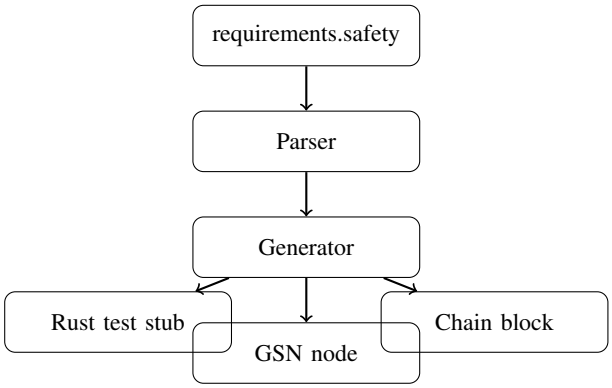


Fig. 1. SCC compilation pipeline. One requirements file generates three artefacts, all derived from the same `Requirement` struct.

VI. Discussion

A. Relation to DO-326A

DO-326A [8] requires a security risk assessment and a plan for security aspects of certification (PSecAC). SCC provides evidence for the *safety* side of the assessment: each requirement is compiled into a test and a chain block, and the chain is anchored externally (via `rnfc-anchor`). The security requirements (SEC-01 to SEC-13) are tracked separately in the project's `SECURITY_REQUIREMENTS.md`.

B. Open Problems

Three problems remain open: (i) a formal semantics for the DSL, so that the generated tests can be proven equivalent to the requirements; (ii) composition, so that multiple safety cases can be combined; (iii) integration with runtime monitors, so that the compiled tests are not only stubs but active monitors on the drone.

VII. Conclusion

We presented SCC, a tool that compiles safety requirements into executable evidence: Rust tests, GSN nodes, and hash chain blocks. The key property is single source of truth: changing one line in the requirements regenerates all artefacts and re-validates the chain. SCC is implemented in Rust, tested with 10 unit tests, and evaluated on a 3-requirement UAV envelope. Future work includes formal semantics, composition, and runtime monitoring.

References

[1] T. Kelly, "Arguing safety: A systematic approach to managing safety cases," in *PhD Thesis, University of York*, 1998.
[2] T. Kelly and R. Weaver, "The goal structuring notation – a safety argument notation," *Proceedings of DSN 2004*, 2004.
[3] EASA, "Special condition for small-category vtol aircraft," European Union Aviation Safety Agency, Tech. Rep., 2019.
[4] RTCA, "Do-178c: Software considerations in airborne systems," RTCA, Inc., Tech. Rep., 2011.
[5] E. Bartocci, Y. Falcone, A. Francalanza, and G. Reger, "Introduction to runtime verification," *Lectures on Runtime Verification*, 2018.
[6] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *USENIX Security Symposium*, 2009.
[7] R. C. Merkle, "A digital signature based on a conventional encryption function," in *CRYPTO*, 1988.
[8] RTCA, "Do-326a: Airworthiness security process specification," RTCA, Inc., Tech. Rep., 2014.